A Private Zcash Payment, End to End

What is constructed, what is proved, and what the ledger checks

m@rek.onl

One payment, in indivisible units

The *ledger* is the accepted record of value transfers. A *transaction* proposes a change to that record. One *zatoshi* is 10^{-8} ZEC; amounts are integers.

$$\underbrace{80{,}000}_{\text{existing value}} = \underbrace{50{,}000}_{\text{payment}} + \underbrace{20{,}000}_{\text{change}} + \underbrace{10{,}000}_{\text{fee}}.$$

- A *note* privately describes an amount and receiving capability.
- *Change* is an output returned to the sender.
- The *fee* is publicly accounted value available to the producer who includes the transaction.

Create two new notes without publicly identifying the old note. This is checked payment arithmetic, not a generated transaction.

A private note needs a public representative

A *commitment* conceals a message while binding its creator to that message, under the construction's security assumptions. The message and randomness together are its *opening*.

$$\mathsf{cm} = \mathsf{NoteCommit}(\mathsf{receiver}, \mathsf{value}, \mathsf{creation\ data}; \mathsf{rcm}).$$

Here rcm is fresh commitment randomness. The ledger stores a public value cmx derived from cm, not the private note.

A *hash* maps data to a fixed-size summary, called a *digest*. Finding different inputs with the same digest is assumed infeasible for the hashes used here.

A hash is not automatically a hiding commitment: hashing a small predictable amount would permit guessing it.

Prove existence without revealing the position

A *hash tree* repeatedly hashes pairs of child values. Its top value, the *root*, commits to the leaves below it. New note commitments are appended as leaves.

A *membership witness* contains the leaf position and the sibling hashes needed to recompute the root. More generally, a *witness* is private data supporting a public claim.

$$\text{private old commitment} \longrightarrow \text{private path} \longrightarrow \text{public anchor}.$$

An *anchor* is an eligible earlier root. A *proof* lets a verifier check a claim about a witness; the proof system used here is designed not to reveal that witness.

Each *shielded pool* maintains its own note tree, spent-tag set and public aggregate balance.

Prevent a second spend without removing a leaf

Removing the old commitment would identify the spent note. Instead, publish a *nullifier*: its deterministic spend tag.

$$\mathsf{nf} = \mathsf{Nullifier}(\mathsf{nk}, \text{old note}).$$

The function uses a private nullifier key nk and the note's committed creation data.

- The same note and key give the same tag.
- The proof connects the tag to the privately identified note.
- The ledger rejects a repeated tag within that pool.

The tag is designed not to reveal which commitment was spent. The tree keeps its old leaf; the ledger adds a spent tag.

Functional interfaces here use the security assumptions; the companion books specify the complete relations and exceptional cases.

Receiving, viewing and spending are different permissions

A *signature* is an approval of a message made with a secret signing key and checked using its public verification key.

Capability	What it permits
Public receiver	Construct an output for the recipient.
Incoming viewing key	Discover and reconstruct matching incoming notes.
Full viewing key	Also compute known notes' nullifiers and derive outgoing recovery keys.
Outgoing viewing key	Recover outputs whose sender actually enabled recovery with that key.
Spending key	Approve spending; viewing alone does not grant this.

A valid receiver does not prove that anyone possesses its spending key.

Just enough algebra to explain delivery and balance

The protocol uses a *group*: a set of points with addition, a zero point and subtraction, obeying the usual addition rules. A *scalar* is a number multiplying a point:

$$[a]P = \underbrace{P + \cdots + P}_{a \text{ additions}}, \qquad [a+b]P = [a]P + [b]P.$$

There are r points, with r a large prime: divisible only by 1 and itself. Scalar sums wrap around at r. Recovering a from P and $[a]P$ is assumed hard for suitable points.

A receiver contains a public *diversifier* d, an address-variation identifier, and a public transmission point $\mathsf{pk_d}$:

$$\mathsf{g_d} = \mathrm{DiversifyHash}(d), \qquad \mathsf{pk_d} = [\mathrm{ivk}]\mathsf{g_d}.$$

DiversifyHash deterministically derives the public base point $\mathsf{g_d}$. The incoming viewing scalar ivk remains private.

Deliver the note, then check what arrived

For a fresh temporary scalar esk, the sender publishes $\mathsf{epk} = [\mathsf{esk}]\mathsf{g}_\mathsf{d}$ and obtains the shared point

$$S = [\mathsf{esk}]\mathsf{pk}_\mathsf{d} = [\mathsf{esk}][\mathsf{ivk}]\mathsf{g}_\mathsf{d} = [\mathsf{ivk}]\mathsf{epk}.$$

The recipient computes the same point using ivk. A prescribed hash derives an encryption key from S and epk.

- Encryption turns note-reconstruction data and a *memo*, a sender-chosen message, into a *ciphertext*, readable with that key.
- An *integrity tag* detects changes under the same key.
- The recipient decrypts, checks the tag, reconstructs the note, and checks its derived temporary key and published commitment.

A valid payment proof does *not* guarantee delivery: a malicious sender can attach unrelated ciphertexts.

Our payment uses two Actions

An *Action* pairs one old side with one new side. A *bundle* collects one pool's Actions and their shared data.

Action	Old value	New value	Net value
Real input to payment	80,000	50,000	+30,000
Dummy input to change	0	20,000	−20,000
Total	80,000	70,000	+10,000

A *dummy input* has zero value and need not demonstrate old membership, but still supplies a nullifier and a spend signature. The public count does not identify which input is the dummy.

Each new note's creation identifier is its paired old nullifier. The pairing is protocol structure, not a public input-to-output link.

Hide net values and randomise signing keys

Let V, R, G be prescribed public base points whose relative multipliers are not known. For Action i, choose fresh scalar randomness rcv_i and commit to its private net value:

$$d_i = v_{\mathrm{old},i} - v_{\mathrm{new},i}, \qquad \mathrm{cv}_i^{\mathrm{net}} = [d_i]V + [\mathrm{rcv}_i]R.$$

Negative net values use group subtraction.

Let ask be the spend-authorising scalar and $\mathsf{ak} = [\mathsf{ask}]G$ its public point. Fresh scalar α_i randomises the signing key:

$$\mathrm{rsk}_i = \mathsf{ask} + \alpha_i, \qquad \mathrm{rk}_i = \mathsf{ak} + [\alpha_i]G = [\mathrm{rsk}_i]G.$$

The public rk_i verifies this Action's spend signature. Value commitments and signing keys serve different purposes.

What the private relation must connect

The public claim names the anchor, nullifier, new commitment, net value commitment and randomised signing key. The witness supplies old/new note data, the old path, viewing-key components and the required randomness.

- The old commitment exists under the anchor when its value is nonzero.
- The old receiver matches the viewing-key components, and the nullifier and randomised signing key follow from those same components.
- The new commitment contains the intended new note data.
- Both amounts satisfy $0 \leq v < 2^{64}$.
- Their difference opens the published net value commitment.
- Nonzero old and new values are allowed by the selected pool rules.

The proof does not itself supply the spending-key signature, check previous spends, or authenticate note delivery.

1. Turn a computation into equations

Arithmetic modulo 97 identifies integers that differ by a multiple of 97. The resulting *field* $\mathbb{F}_{97}$ supports addition, multiplication and division by every nonzero value.

An *arithmetic circuit* is a fixed collection of cells connected by equations. A private witness fills some cells; the public claim fixes others. Consider

$$s^3 + s + 5 = t \quad \text{in } \mathbb{F}_{97}, \qquad s = 3,\ t = 35.$$

Break the computation into rows:

$$s^2 = 9, \qquad s^3 = 27, \qquad s^3 + s = 30, \qquad 30 + 5 = 35.$$

The same private s must be used throughout: equality between reused cells must also be enforced.

This small-field example explains arithmetic, not cryptographic security.

2. A complete four-row circuit

Use columns a, b, c for computed values. Fixed coefficient columns q select the equation on each row; the public column π supplies the claimed result.

$$q_M ab + q_L a + q_R b + q_O c + q_C + \pi = 0.$$

i	a	b	c	q_M	q_L	q_R	q_O	q_C	π
0	3	3	9	1	0	0	-1	0	0
1	9	3	27	1	0	0	-1	0	0
2	27	3	30	0	1	1	-1	0	0
3	30	0	0	0	1	0	0	5	-35

The required equalities are

$$a_0 = b_0 = b_1 = b_2, \qquad c_0 = a_1, \quad c_1 = a_2, \quad c_2 = a_3.$$

Negative entries are residues modulo 97.

3. Columns become polynomials

A *polynomial* is a sum of coefficients times powers of X. *Interpolation* chooses its coefficients to match prescribed values. Use row points $H = \{1, 22, 96, 75\} \subset \mathbb{F}_{97}$:

$$a(X) = 90 + 61X + 22X^2 + 24X^3,$$
$$b(X) = 75 + 32X + 25X^2 + 65X^3,$$
$$c(X) = 65 + 16X + 3X^2 + 22X^3.$$

Interpolate the fixed and public columns too. Their equation becomes a polynomial $g(X)$. The *vanishing polynomial* $Z_H(X) = X^4 - 1$ is zero at every row point.

$$g(X) = (X^4 - 1)(61 + 70X + 43X^2 + 47X^3 + 81X^4 + 46X^5).$$

Every row satisfies the equation exactly when Z_H divides g.

4. Check the quotient at a new point

The *quotient* $q(X)$ is the second factor in $g(X) = Z_H(X)q(X)$. At $x = 20$, outside the row points,

$$Z_H(20) = 46, \qquad q(20) = 67, \qquad g(20) = 75, \qquad 46 \cdot 67 = 75 \pmod{97}.$$

Change only c_0 from 9 to 10. Polynomial division now leaves a nonzero *remainder*:

$$r(X) = 24(1 + X + X^2 + X^3).$$

At 20, the new numerator is 20 while its quotient times Z_H is 64: the identity test rejects.

A nonzero polynomial has at most as many roots as its highest exponent. Fixing the polynomials *before* an unpredictable test point makes a false identity hard to hide in a sufficiently large field.

Halo authenticates the claimed evaluations

A *polynomial commitment* binds a hidden polynomial. An *opening proof* checks its claimed value at a chosen point without sending all its coefficients.

1. Commit to the circuit columns with the prescribed privacy randomness.
2. Prove the equalities between reused cells and membership in required tables.
3. Combine the equations and commit to the quotient's pieces.
4. Derive later test points by hashing the preceding public record.
5. Check authenticated evaluations, ending in a checked group equation.

That ordered record is the *transcript*. Its order prevents choosing earlier commitments after seeing later challenges. A quotient identity with unauthenticated evaluations would not be a proof.

What the proof contract promises

- *Completeness*: honest valid inputs produce accepting proofs, subject to the construction's stated exceptional events.
- *Knowledge soundness*: a sufficiently successful prover can be converted, under the theorem's conditions, into an algorithm recovering a satisfying witness.
- *Zero knowledge*: the verifier's complete view can be simulated from public inputs alone.

These are complete proof-system claims under stated assumptions. Commitment binding alone does not imply witness recovery. Separately random-looking messages do not establish privacy of their joint transcript.

The private relation is one part of transaction validity, not a substitute for the ledger's other checks.

Conservation needs a binding signature

For public pool withdrawal b, form the *binding verification key*

$$\mathsf{bvk} = \sum_i \mathsf{cv}_i^{\mathrm{net}} - [b]V.$$

When the net private values sum to b, the value terms cancel:

$$\mathsf{bvk} = [\mathsf{bsk}]R, \qquad \mathsf{bsk} = \sum_i \mathsf{rcv}_i \pmod{r}.$$

The *binding signature* approves the transaction digest using this aggregate randomness and base R. For our payment,

$$30{,}000 - 20{,}000 - 10{,}000 = 0, \qquad \mathsf{bvk} = [\mathsf{rcv}_0 + \mathsf{rcv}_1]R.$$

Spend signatures approve use of individual spending authority. The binding signature enforces aggregate conservation together with the proof and signature assumptions. Amount and transaction-size bounds exclude wraparound by a multiple of r.

The signer must approve the user's payment

The signature message is a prescribed digest of transaction data, not a wallet's informal label for that data.

- The constructor selects the notes, recipients, change, fee and randomness.
- The prover receives the private witness and therefore learns it.
- The signer checks the intended recipient and amount, independently recognises change, and approves the exact effects.
- The completed transaction is checked again before submission.

The approved effects include the ciphertexts: delivery data cannot simply be replaced after signing. A valid signature does not show that a user interface described those effects honestly.

The node checks the whole transaction

A *node* is a participating computer. A validating node checks the applicable rules, not merely one proof.

1. Decode the transaction, enforce allowed operations and amount limits.
2. Verify its proof against the correct public inputs and the circuit's prescribed verification data.
3. Recompute the signature digest; verify spend and binding signatures.
4. Check eligible anchors and unused nullifiers, including duplicates inside this transaction and the containing block.
5. Validate any other transaction components and validity deadlines.
6. Check the public fee and resulting pool balances.

For our all-shielded payment, the public withdrawal is the fee: $b = f = 10{,}000$. Accepting it appends two commitments and inserts two nullifiers.

A block gives the transaction its context

A *block* contains an ordered transaction list. Its *header* is a public summary naming the previous block by hash and committing to the block's data.

- A *branch* follows those parent links from the prescribed starting block. Its ledger state belongs to that branch.
- A node validates the complete body and its header commitments, not an isolated list of transaction identifiers.
- It checks the block's permitted new issuance, fees and allocations.
- Transactions are checked against branch-local state, including changes from earlier transactions where the rules permit them.

A hash authenticates data; it does not validate the arithmetic or authorisation inside that data.

Confirmation is confidence in a branch

Proof of work is a publicly checkable result of computational search. Zcash requires both its prescribed search solution and a header hash no larger than the required difficulty target.

Work is the numerical score assigned from that target. A node selects a visible, fully validated branch with the greatest accumulated work, not merely the greatest number of blocks.

- Work does not excuse invalid transactions or a missing block body.
- More work after a payment makes replacing its branch more demanding.
- A competing valid branch can still remove a previously mined payment.

Confirmation confidence depends on assumptions about adversarial computing power and network communication; it is not absolute finality.

Discovery can use less than a full transaction

A *compact block* is a service's reduced representation for wallet scanning, not a different block validated by nodes. It carries nullifiers, new commitments, temporary public keys and a prefix of each incoming ciphertext.

- Incoming viewing keys discover candidate receipts.
- The wallet reconstructs each candidate note and checks its derived temporary key and commitment.
- The compact prefix omits the memo and integrity tag. This is not full ciphertext authentication.
- A full viewing key also computes known notes' nullifiers and matches them against observed spends.

Successful reconstruction identifies relevant data. It does not prove that the server supplied a valid or complete chain.

Discovery is not yet spendability

To spend a discovered note, retain its content, pool, branch, leaf position and path to an eligible anchor.

- Every Action appends a commitment, even if this wallet detects no receipt or the output has zero value.
- After n earlier commitments, the next c outputs occupy positions $n, \ldots, n + c - 1$.
- Later appends change the sibling hashes needed by owned notes.
- A small append summary suffices to compute future roots, but cannot recover arbitrary old membership paths.

Retain the additional tree data needed for owned notes, and tie saved states to block identities.

A root supplied by an untrusted server is not its own authentication.

A reorganisation changes more than confirmations

A *reorganisation* switches the selected branch. Suppose the shared ancestor has 11 commitments. Our outputs occupy positions $11, 12$ on one branch.

If a replacement branch adds three others before including the same payment, its outputs instead occupy $14, 15$.

- Roll back received-note locations, observed spends, tree state, retained paths and confirmations together.
- Make corresponding database changes all visible together, or none of them.
- Preserve the user's intention and retained transaction separately; recheck whether the payment is valid, conflicted or expired.

Re-labelling an old path with a new height does not make it valid.

The privacy boundary includes the wallet service

- A chain observer sees Action counts, pools, public balances, anchors, nullifiers, randomised public keys, commitments and ciphertexts.
- A recipient learns its note and sender-chosen memo, not a cryptographic guarantee of the sender's real-world identity.
- Viewing-key disclosure grants the corresponding viewing capability.
- A wallet service can observe requests, connection metadata, relevant-transaction retrievals and broadcasts.
- A server can omit relevant data; note reconstruction alone does not establish completeness or best-branch selection.

Zero knowledge protects the proof's private witness from its verifier. It does not erase public inputs, network observations or information deliberately shared with a prover.

One payment, several distinct responsibilities

user-approved recipient, change and fee

↓

notes, commitments, nullifiers and encrypted delivery

↓

private proof, spend signatures and binding signature

↓

transaction checks, block checks and branch selection

↓

wallet discovery, retained paths and reversible state.

No single accepting proof replaces the rest.

The Zcash Arboretum supplies the complete constructions, source pins and executable examples.